STUDENT PERFORMANCE PREDICTION (MACHINE LEARNING)

A PROJECT REPORT

SUBMITTED BY

Roll Number	Registration Number	Student Code	Student Name
24103460111	24103002893	BWU/BCA/24/117	ARITRA MONDAL

Of in partial fulfillment for the award of the degree

BACHELOR OF COMPUTER APPLICATION

IN

DEPARTMENT OF COMPUTATIONAL SCIENCE



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ABSTRACT

The Student Performance Prediction System is a Python-based machine learning project designed to estimate a student's final marks using selected academic and lifestyle-related attributes. The project demonstrates the practical use of supervised learning and data analysis to study relationships between student characteristics and academic outcomes.

The system uses seven input features: Attendance, Previous_Marks, Assignment_Score, Study_Hours, Participation, Sleep_Hours and Internet_Hours. Final_Marks is used as the prediction target. The dataset used for this experiment contains 200 simulated student records prepared for academic demonstration. The data are loaded from a CSV file, checked for duplicates and missing values, and separated into input features and target values.

A Random Forest Regressor containing 200 estimators is trained using an 80:20 train-test split. The model is evaluated using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and the coefficient of determination (R^2).

The experiment produced MAE = 6.385, MSE = 65.0405, RMSE = 8.065 and R^2 = 0.3780 on 40 held-out test records. These results describe only this experiment and should not be treated as proof of accuracy on real student records.

The project also generates visual reports, including a final-marks distribution, feature-importance chart and actual-versus-predicted comparison. The trained model can be saved and reused for new predictions. The report describes the introduction, literature survey, system analysis and design, implementation, outputs, testing, conclusion and future enhancements.

INDEX

Section Number	Section Title	Page No.
-	ABSTRACT	2
-	LIST OF FIGURE	3
-	LIST OF TABLE	3
-	LIST OF SCREENSHOTS	3
1	INTRODUCTION	4
2	LITERATURE SURVEY	5
3	SYSTEM ANALYSIS AND DESIGN	6
4	IMPLEMENTATION	14
5	OUTPUT SCREENSHOT	15
6	TESTING AND DEBUGGING	18
7	CONCLUSION AND FUTURE ENHANCEMENT	20
8	REFERENCES	21

List of Figures

- Fig. 3.1 System architecture
- Fig. 3.2 Model workflow
- Fig. 5.1 Final-marks distribution
- Fig. 5.2 Feature importance
- Fig. 5.3 Actual versus predicted marks

List of Tables

- Table 3.1 Dataset attributes
- Table 3.2 System requirements
- Table 3.3 Model configuration
- Table 6.1 Test cases

List of Screenshots

- Dataset preview
- Model evaluation output
- Generated analytical charts

1. INTRODUCTION

Academic performance is influenced by several factors, including attendance, previous achievement, assignment performance, study habits and participation. Understanding how these factors relate to final marks can help demonstrate the role of data analysis in education. Traditional review often examines each measure separately and may not reveal patterns across several variables.

The Student Performance Prediction System is developed as an academic machine learning prototype. It accepts structured student attributes, learns a relationship between the selected features and final marks, and estimates marks for unseen records. The project uses Python and commonly used data-science libraries to create a reproducible workflow.

1.1 Background

Machine learning is a branch of artificial intelligence that enables computer systems to learn patterns from examples. In supervised learning, a model is trained using records that contain both input variables and known outcomes. Regression is suitable when the outcome is numerical, such as examination marks.

1.2 Project Overview

The application reads a CSV dataset containing 200 simulated student records. It performs basic preprocessing, separates the seven input attributes from the target column, divides the data into training and testing subsets, trains a Random Forest Regressor and evaluates the results.

1.3 Need for the Project

The project provides practical experience in data preparation, model training, performance measurement and visualization. It also demonstrates why a model must be evaluated on data that were not used during training and why dataset quality is essential.

1.4 Organization of the Report

The following sections review related concepts, describe system requirements and architecture, explain the methodology and implementation, present generated outputs, document test cases and discuss limitations and future enhancements.

2. LITERATURE SURVEY

Student performance analysis is an area of educational data mining and learning analytics. These fields apply statistical and computational methods to educational records to identify patterns, understand engagement and support evidence-based decisions. The usefulness of any analysis depends on the quality, representativeness and appropriate use of the data.

2.1 Educational Data Mining

Educational Data Mining studies information generated in learning environments. Common tasks include classification of performance levels, prediction of academic outcomes, clustering of learner groups and analysis of engagement. Such techniques can support research and exploratory analysis, but they should not replace professional judgement.

2.2 Supervised Learning and Regression

Supervised learning uses labelled examples to learn a mapping from input features to a target. Regression methods estimate continuous numerical values. In this project, the input features are student attributes and the target is Final_Marks.

2.3 Random Forest Algorithm

A Random Forest is an ensemble of decision trees. Each tree is trained using a randomized sample of data and features. For regression, the outputs of the trees are combined, usually by averaging. This approach can model nonlinear relationships and reduce dependence on the decisions of a single tree.

2.4 Model Evaluation

MAE measures average absolute error, MSE averages squared errors, RMSE expresses error in the target unit and R^2 compares the model with a baseline that predicts the mean target. No single metric fully describes model quality, so multiple measures should be considered together.

2.5 Summary

The concepts reviewed provide the theoretical basis for using a Random Forest regression model. The present experiment is a demonstration using simulated records; its results require validation on representative real-world data before any practical deployment.

3. SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

System analysis identifies the problem, required inputs, processing steps and expected outputs. In a manual approach, student information may be reviewed using spreadsheets or separate records. A machine learning workflow can combine multiple attributes in a single model and produce repeatable estimates for analysis.

Existing System

A conventional manual approach may review attendance, previous marks and assignment scores separately. It can be difficult to consistently combine several factors, compare many records and estimate outcomes for new cases. Spreadsheet calculations can summarize data but do not automatically learn complex predictive relationships.

Problems in the Existing System

- Manual analysis of multiple student attributes can be time-consuming.
- Relationships between several variables may be difficult to identify by inspection.
- Repeated calculations may lead to inconsistent results.
- There may be no consistent evaluation procedure for predictions.
- A model trained on poor-quality data may produce misleading results.

Proposed System

The proposed system loads a structured CSV file, checks the records, selects the input features and target, splits the data, trains a Random Forest Regressor, evaluates predictions and produces charts. The trained model can also be saved for later use.

3. SYSTEM ANALYSIS AND DESIGN

3.2 System Design

System design defines the main components of the project and the order in which data move through them. The proposed solution is organized into data input, preprocessing, model training, evaluation and output stages.

3. SYSTEM ANALYSIS AND DESIGN

3.2 System Design

System design defines the main components of the project and the order in which data move through them. The proposed solution is organized into data input, preprocessing, model training, evaluation and output stages.

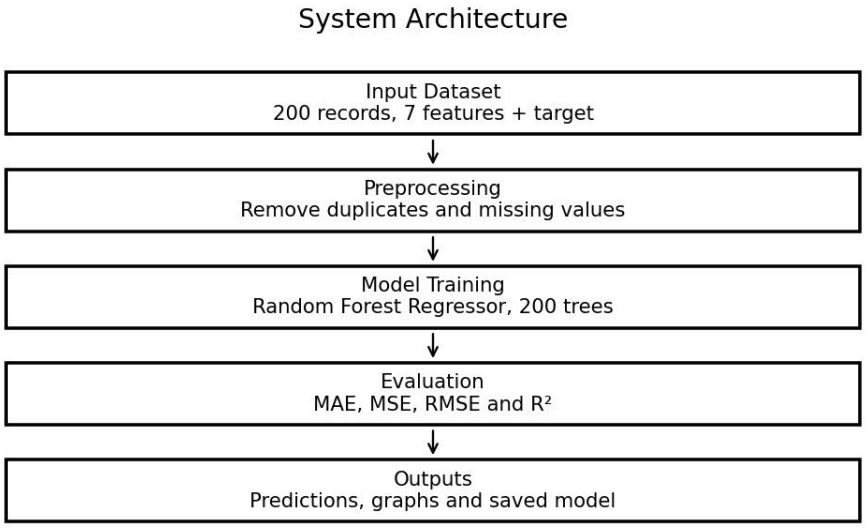


Fig. 3.1: System Architecture of Student Performance Prediction

Module	Main responsibility
Dataset Input	Reads the CSV file and checks the required columns.
Preprocessing	Removes duplicate and incomplete records.
Feature Selection	Separates seven input attributes and Final_Marks.
Model Training	Fits a Random Forest Regressor on training data.
Evaluation	Calculates regression metrics on the test subset.
Output Generation	Creates charts and saves the trained model.

3. SYSTEM ANALYSIS AND DESIGN

3.3 Dataset Attributes

The model uses seven input variables and one output variable. Each row represents one simulated student record. The values are numeric so that they can be processed by the regression algorithm.

Attribute	Description	Role
Attendance	Attendance percentage	Input
Previous_Marks	Previous academic marks	Input
Assignment_Score	Assignment performance score	Input
Study_Hours	Approximate daily study hours	Input
Participation	Class participation score	Input
Sleep_Hours	Approximate daily sleep hours	Input
Internet_Hours	Approximate daily internet-use hours	Input
Final_Marks	Final academic marks	Target

Data Preparation

The dataset is stored in CSV format. pandas is used to read the file and select the required columns. Duplicate rows and rows with missing values are removed before training. The feature matrix X contains the seven input columns, while the target vector y contains Final_Marks.

Dataset Note

The 200 records used in this report are simulated data prepared for project demonstration. They are not actual university student records. The values should not be represented as observations collected from real students.

3. SYSTEM ANALYSIS AND DESIGN

3.4 Model Configuration

Parameter	Value	Purpose
Algorithm	Random Forest Regressor	Predicts a numerical target
Number of estimators	200	Number of decision trees
Train-test split	80:20	Separates training and evaluation data
Random state	42	Supports reproducibility
Input features	7	Student attributes used for prediction
Target	Final_Marks	Value estimated by the model

Core Concepts Used

Concept	Meaning	Use in project
Regression	Predicting a numeric value	Estimates final marks
Decision Tree	Sequence of feature-based splits	Base learner in the forest
Ensemble Learning	Combining multiple models	Random Forest
Train-test split	Separating fit and evaluation data	Checks unseen-record performance
Feature importance	Relative contribution in model splits	Ranks input features
Model persistence	Saving a trained estimator	Reuse without retraining

3. SYSTEM ANALYSIS AND DESIGN

3.5 System Flow

The workflow starts when the program reads the dataset. It then prepares the feature and target arrays, divides the records into training and testing subsets, fits the model and evaluates the predictions.

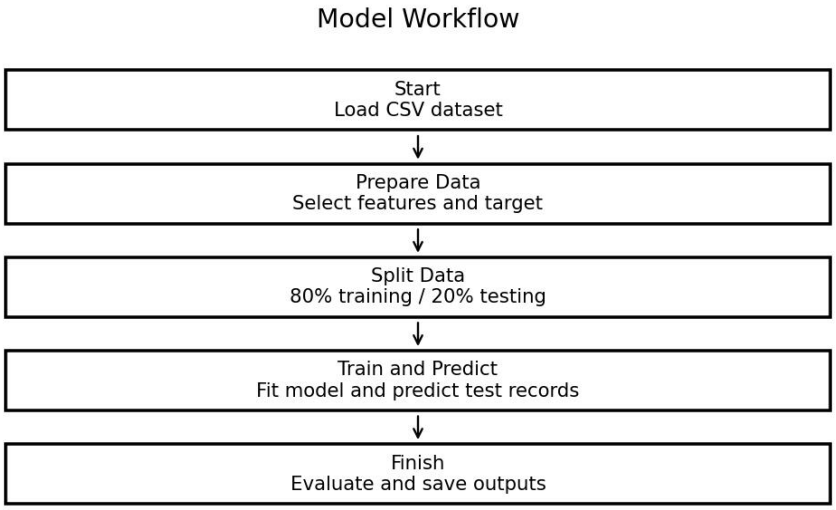


Fig. 3.2: Model Training and Prediction Workflow

Step	Action	Output
1	Load CSV file	DataFrame
2	Clean duplicates and missing rows	Prepared dataset
3	Select features and target	X and y
4	Split dataset 80:20	Training and test sets
5	Train Random Forest	Fitted model
6	Predict test records	Predicted marks
7	Calculate metrics and charts	Evaluation results
8	Save trained model	Model file

3. SYSTEM ANALYSIS AND DESIGN

3.6 Data Flow Diagram

A Data Flow Diagram (DFD) represents how data move through the system. The student performance dataset is the input, the model-training process performs preprocessing and prediction, and the evaluation stage produces metrics and visual outputs.

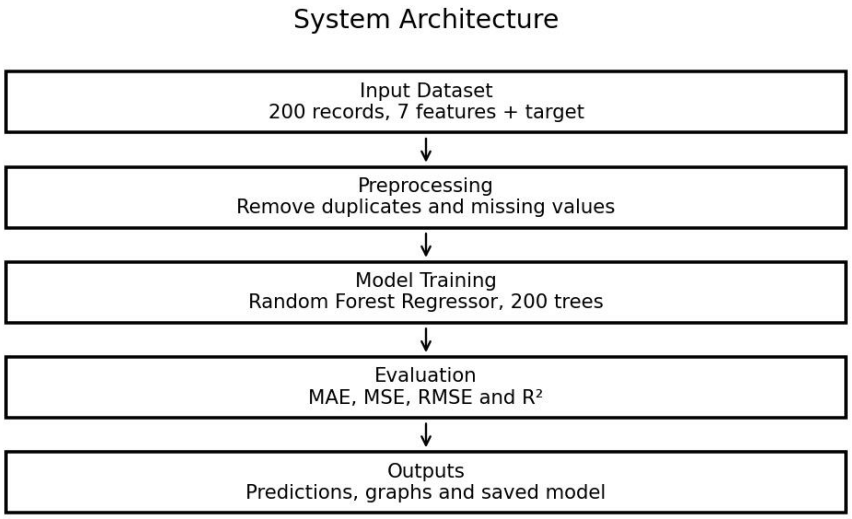


Fig. 3.3: Context-Level Data Flow of the Prediction System

Main Data Flows

- Dataset file to preprocessing: raw student attributes and target marks.
- Preprocessing to model: cleaned feature matrix and target vector.
- Training to prediction: fitted Random Forest model and test features.
- Prediction to evaluation: actual and predicted marks.
- Evaluation to output: error metrics, graphs and saved model.

3. SYSTEM ANALYSIS AND DESIGN

3.7 System Requirements

Hardware	Requirement
Processor	Modern dual-core processor or better
RAM	4 GB minimum; 8 GB recommended
Storage	At least 500 MB free space
Display	Standard laptop or desktop display

Software / Library	Purpose
Windows or Linux	Operating system
Python 3.x	Programming language
pandas	CSV loading and data manipulation
NumPy	Numerical operations
scikit-learn	Model training and evaluation
Matplotlib	Graph generation
Joblib / pickle	Saving and loading the model

3.8 Feasibility

The project is technically feasible because it uses open-source Python libraries and can run on a standard computer. It is economically feasible for an academic prototype because the required software is freely available. Operationally, the workflow can be executed from a terminal and extended later with a graphical interface.

3. SYSTEM ANALYSIS AND DESIGN

3.9 Methodology

- Load the 200-record CSV dataset using pandas.
- Remove duplicate records and rows with missing required values.
- Select Attendance, Previous_Marks, Assignment_Score, Study_Hours, Participation, Sleep_Hours and Internet_Hours as features.
- Select Final_Marks as the target variable.
- Split the data into 80% training and 20% testing using random_state=42.
- Train RandomForestRegressor with 200 estimators.
- Predict marks for the 40 test records.
- Calculate MAE, MSE, RMSE and R^2.
- Generate analysis charts and save the model.

3.10 Evaluation Measures

MAE is the average absolute difference between actual and predicted marks. MSE is the average squared difference. RMSE is the square root of MSE and is expressed in marks. R^2 measures performance relative to predicting the mean target value. These measures provide complementary information about prediction error and model fit.

4. IMPLEMENTATION

The implementation phase converts the system design into a working Python program. pandas is used to load and prepare the dataset, scikit-learn provides the Random Forest algorithm and evaluation functions, and Matplotlib generates visualizations.

4.1 Project Structure

File / Folder	Purpose
dataset/student_performance.csv	Input dataset containing 200 simulated records
main.py	Preprocessing, model training, evaluation and prediction
model/student_model.pkl	Saved trained model
graphs/	Generated charts

4.2 Implementation Process

The program reads the CSV file, removes duplicates and incomplete records, then selects the feature columns and target column. The train_test_split function creates the training and testing subsets. The Random Forest model is fitted using the training subset and predicts Final_Marks for the test subset.

4.3 Model Saving

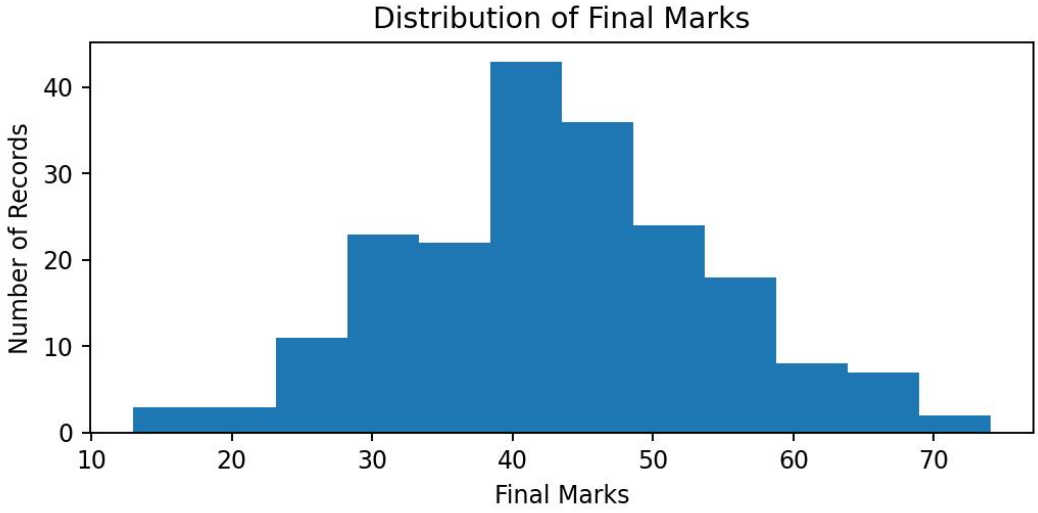
After training, the estimator can be saved using joblib or pickle. The saved model can be loaded later for new predictions. The input values must follow the same feature order and compatible data types used during training.

4.4 Error Handling

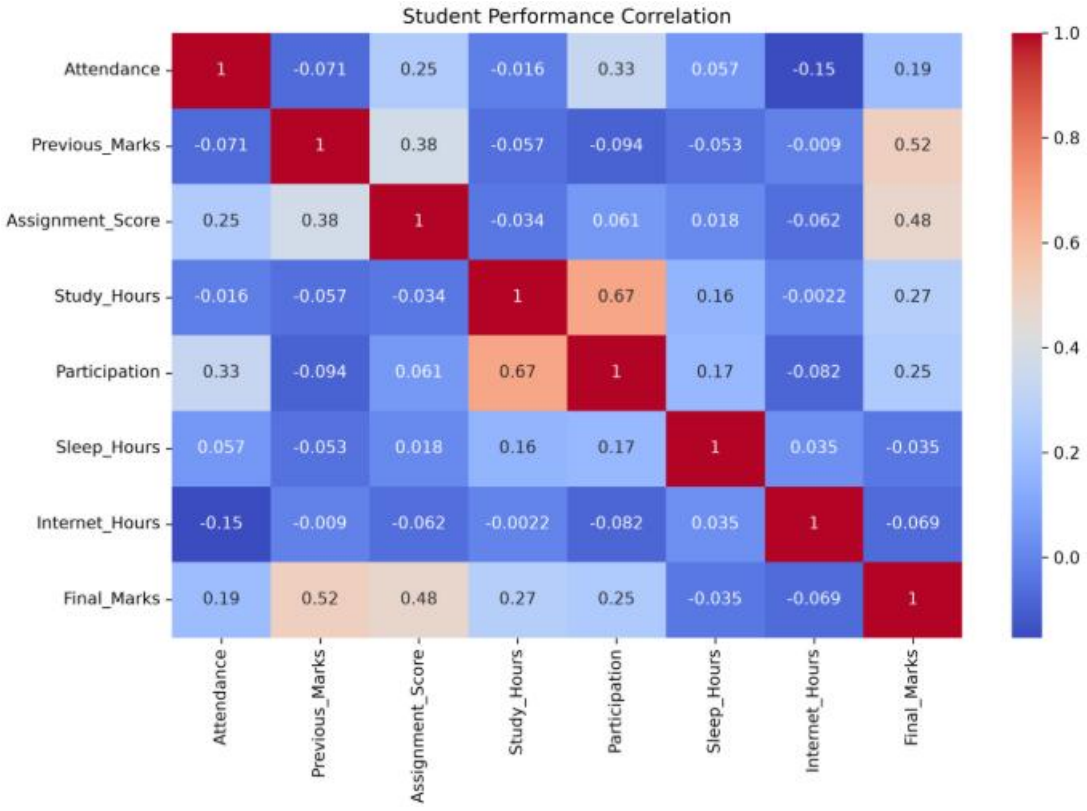
- Check that the CSV file exists at the expected path.
- Verify that all required column names are present.
- Ensure feature and target values are numeric.
- Create model and graph folders before saving output files.

5. OUTPUT SCREENSHOT

The following figures show the analytical outputs generated by the project using the 200-record simulated dataset. They are model-generated charts rather than screenshots of a deployed graphical application.

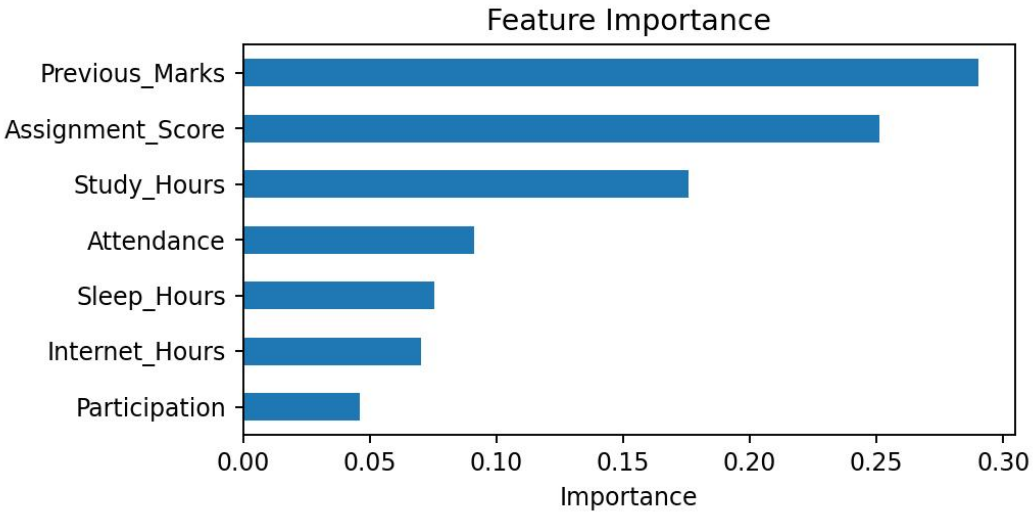


5.1: Distribution of Final Marks



5.2: Feature Correlation Matrix

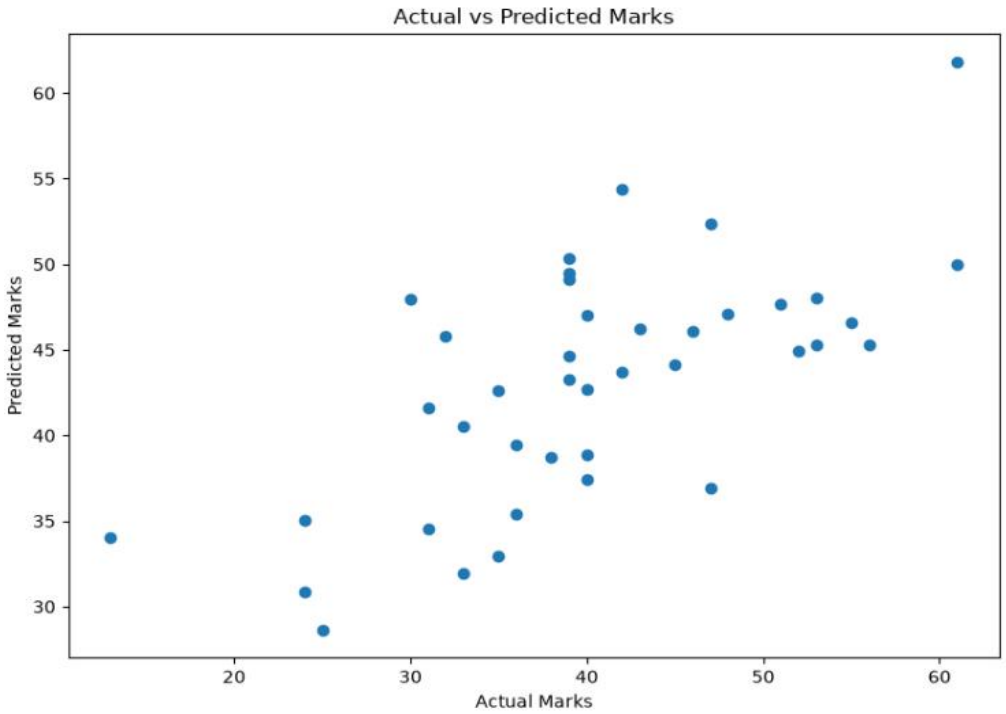
5. OUTPUT SCREENSHOT



5.3: Random Forest Feature Importance

The feature-importance chart displays the relative importance assigned by the fitted Random Forest model to each input attribute. It describes the model's internal split-based importance and does not establish that a feature causes changes in final marks.

5. OUTPUT SCREENSHOT



5.4: Actual Versus Predicted Final Marks

Model Evaluation Output

Metric	Result
MAE	6.3854
MSE	65.0405
RMSE	8.0648
R^2	0.3780

The diagonal reference line indicates where predicted marks would equal actual marks. Points closer to this line represent smaller prediction errors.

6. TESTING AND DEBUGGING

Introduction

Testing and debugging are important stages in the development of the Student Performance Prediction System. Testing checks that data loading, reprocessing, training, evaluation and output generation work as expected. Debugging helps identify and correct issues such as incorrect file paths, missing columns and invalid data types.

Types of Testing

1. Data Validation Testing

The datasets is checked for the required eight columns. Duplicate records are removed and rows with missing required values are excluded from the modelling workflow.

2. Model Training Testing

The training procedure is executed using the 80% training subset. The fitted Random Forest model is then used to generate predictions for the held-out test subset.

3. Evaluation Testing

The program calculates MAE, MSE, RMSE and R^2 from the actual and predicted values. The values are checked to ensure that the evaluation functions return numeric results.

4. Output Testing

The program generates the final-marks distribution, feature-importance and actual-versus-predicted charts. The trained estimator is saved to the model folder.

6. TESTING AND DEBUGGING

TEST CASES

Test ID	Description	Input / Action	Expected Result	Status
TC-1	Load dataset	Read CSV file	Dataset loads successfully	Executed
TC-2	Check columns	Validate 8 required columns	All columns are available	Executed
TC-3	Clean records	Remove duplicates and missing rows	Clean data prepared	Executed
TC-4	Split dataset	80:20 split, random_state=42	160 train / 40 test	Passed
TC-5	Train model	Fit 200-tree Random Forest	Model trains successfully	Passed
TC-6	Generate predictions	Predict test subset	40 predictions returned	Passed
TC-7	Calculate metrics	Compare actual and predicted	Four metrics calculated	Passed
TC-8	Save charts/model	Write outputs to folders	Files generated	Configured

Debugging Considerations

- FileNotFoundError: confirm the dataset folder and CSV filename.
- KeyError: verify that the column names match the CSV headers.
- ValueError: check for non-numeric values in the model columns.
- Output path error: create the model and graphs directories before saving.

7. CONCLUSION AND FUTURE ENHANCEMENT

The Student Performance Prediction System demonstrates a complete supervised machine learning workflow for estimating final marks from selected student attributes. The project covers dataset loading, data cleaning, feature selection, train-test splitting, Random Forest training, model evaluation, visualization and model saving.

The experiment used 200 simulated records, with 160 records for training and 40 records for testing. The observed evaluation results were MAE 6.3854, MSE 65.0405, RMSE 8.0648 and R^2 0.3780. These values describe the current test split only and should not be interpreted as evidence of performance on real student populations.

Future Enhancement

- Collect a larger, diverse dataset of real student records only with appropriate institutional permission and privacy safeguards.
- Validate the model using cross-validation and a separate external test dataset.
- Compare Random Forest with Linear Regression, Decision Tree and Gradient Boosting.
- Build a user-friendly web interface for entering student attributes and viewing predictions.
- Add explainability tools and uncertainty estimates to make predictions easier to interpret.
- Review fairness, privacy and security before any real-world deployment.

The current project is an academic prototype. Predictions should not be used as the sole basis for academic decisions.

8. REFERENCES

1. Python Software Foundation. Python Documentation. https://docs.python.org/3/
2. pandas Development Team. pandas Documentation. https://pandas.pydata.org/docs/
3. scikit-learn Developers. RandomForestRegressor Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html
4. scikit-learn Developers. Model Evaluation: Regression Metrics. https://scikit-learn.org/stable/modules/model_evaluation.html
5. Matplotlib Development Team. Matplotlib Documentation. https://matplotlib.org/stable/
6. NumPy Developers. NumPy Documentation. https://numpy.org/doc/
7. Breiman, L. (2001). Random Forests. Machine Learning, 45, 5–32.
8. James, G., Witten, D., Hastie, T. and Tibshirani, R. An Introduction to Statistical Learning. Springer.
9. Project dataset: student_performance_200.csv. Simulated dataset prepared for academic demonstration (200 records).
10. Class notes and study materials provided by Brainware University.